DATABASE DESIGN DOCUMENT

Motorcycle Shop E-Commerce System

MongoDB Collections and Schema Reference

Derived from FR1.1 to FR1.10 | MERN Stack (MongoDB, Express.js, React, Node.js)

PREPARED BY

Benitez, Alwin C.
Roduta, Kelly Laurence Q.

SECTION

BSIT – S – 3A

SUBMITTED TO

Mr. Dalisay

DATE

September 2026

Contents

1. Design Principles 3

2. Collections Overview 4

3. Relationships and Keys 4

3.1 Relationship Table 4

3.2 Entity Relationship Diagram 6

4. Collection Specifications 7

4.1 users 7

4.2 addresses 8

4.3 categories 8

4.4 brands 9

4.5 products 9

4.6 carts 10

4.7 orders 10

4.8 installment_plans 12

4.9 payment_schedules 12

4.10 payments 13

4.11 reviews 14

4.12 suppliers 14

4.13 restocks 15

4.14 inventory_transactions 15

4.15 notifications 16

4.16 settings 16

5. Aggregation Map 17

6. Normalization Notes 18

7. Implementation Notes 18

1. Design Principles

This document describes the MongoDB collections needed to support the Motorcycle Shop E-Commerce System. It is a reference to follow while building the Mongoose models and API, written from the functional requirements (FR1.1 to FR1.10).

1. **No computed fields are stored.** Line totals, order totals, tax, average ratings, remaining balances, financed amount, monthly payment and total expenses are derived with aggregation (`$multiply`, `$sum`, `$avg`, `$lookup`). Section 5 maps each one.
2. **Snapshots are not computed fields.** Values that must stay frozen after an order is placed (unit price, product name, installment plan terms) are copied into the order. A later price or plan edit must not change old orders.
3. **Embed or reference deliberately.** Data that is always read with its parent and has no life of its own (order items, cart items, addresses, images) is embedded. Anything shared, unbounded or queried on its own (payments, schedule rows, reviews) is a separate collection linked by `ObjectId`.
4. **Timestamps everywhere.** Every collection uses `timestamps: true`, which adds `createdAt` and `updatedAt`. These are not repeated in the tables below. `_id` is implicit.
5. **Archive instead of delete** for records that other documents reference (products, plans, suppliers, users), using `isArchived` or `isActive` flags.
6. **Keys.** Every collection has an `_id` primary key (PK). A foreign key (FK) is an `ObjectId` field that stores the `_id` of a document in another collection. MongoDB does not enforce foreign keys, so the application enforces them. Section 3 and every collection in Section 4 list the keys.

2. Collections Overview

#	Collection	Purpose	Requirements
1	`users`	Accounts, roles, profile, ID documents, password reset	FR1.1, FR1.8
2	`addresses`	Multiple saved shipping addresses per user	FR1.1.1, FR1.5.2
3	`categories`	Product categories	FR1.3
4	`brands`	Motorcycle and parts brands	FR1.3
5	`products`	Motorcycles, parts, accessories, riding gear	FR1.2
6	`carts`	One persistent cart per user	FR1.4
7	`orders`	Orders, shipping, status, installment application, cancellation	FR1.5, FR1.10.3
8	`installment_plans`	Admin-configured plans (term, rate, downpayment, fee)	FR1.10.1
9	`payment_schedules`	One row per installment due date	FR1.10.4, FR1.10.6
10	`payments`	Every payment, downpayment, payoff and manual entry; receipts	FR1.5.5, FR1.10.5, FR1.10.8
11	`reviews`	Ratings and reviews with admin moderation	FR1.7
12	`suppliers`	Supplier details and product lines	FR1.6.2
13	`restocks`	Restocking records and unit costs	FR1.6.1, FR1.6.3
14	`inventory_transactions`	Append-only stock ledger	FR1.6.4, FR1.6.5
15	`notifications`	In-app alerts and Mailtrap email log	FR1.5.10, FR1.6.6
16	`settings`	Tax, low-stock threshold, penalty and default rules	FR1.10.7, FR1.10.9

3. Relationships and Keys

PK = primary key (`_id`). **FK** = foreign key, an `ObjectId` that stores the `_id` of a document in another collection. **UK** = unique index. MongoDB does not enforce foreign keys itself, so each FK is enforced in the application layer (Mongoose `ref`, validation and transactions).

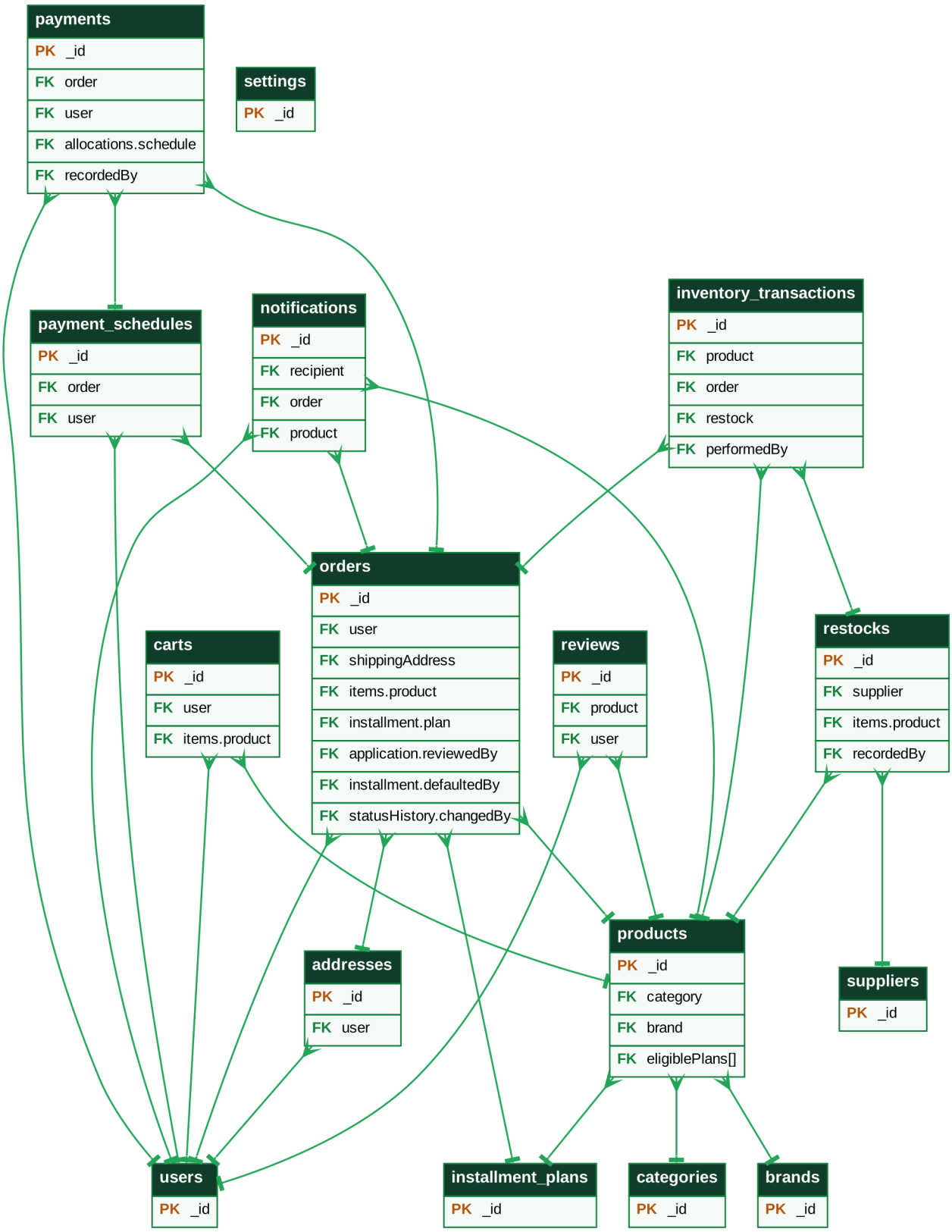
3.1 Relationship Table

From	Cardinality	To	Foreign key (in the To collection)
`users`	1 to 0..1	`carts`	`carts.user`
`users`	1 to many	`orders`	`orders.user`
`users`	1 to many	`addresses`	`addresses.user`
`addresses`	1 to many	`orders`	`orders.shippingAddress`
`users`	1 to many	`reviews`	`reviews.user`
`users`	1 to many	`notifications`	`notifications.recipient`
`categories`	1 to many	`products`	`products.category`
`brands`	1 to many	`products`	`products.brand`

From	Cardinality	To	Foreign key (in the To collection)
products	1 to many	reviews	reviews.product
products	1 to many	inventory_transactions	inventory_transactions.product
products	many to many	installment_plans	products.eligiblePlans
installment_plans	1 to many	orders	orders.installment.plan
orders	1 to many	payment_schedules	payment_schedules.order
orders	1 to many	payments	payments.order
payment_schedules	many to many	payments	payments.allocations.schedule
orders	1 to many	inventory_transactions	inventory_transactions.order
suppliers	1 to many	restocks	restocks.supplier
restocks	1 to many	inventory_transactions	inventory_transactions.restock

3.2 Entity Relationship Diagram

Each box shows the primary key and the foreign keys of one collection. A line ends in a crow's foot on the collection that holds the foreign key (the many side) and in a bar on the collection it points to (the one side). Audit fields (`recordedBy`, `performedBy`, `reviewedBy`, `defaultedBy`, `changedBy`) also point to `users`; their lines are left out so the diagram stays readable.



4. Collection Specifications

Types follow Mongoose conventions. Fields marked **required** or **unique** map directly to schema options. Bracketed types such as `[String]` are arrays. The **Key** column marks the primary key (PK), foreign keys (FK, with the collection they point to) and unique fields (UK). Below each table, a keys box lists the primary key, every foreign key, and which collections point back to it.

4.1 users

Covers: FR1.1.1 to FR1.1.7, FR1.8.1. Stores both Admin and Customer accounts.

Field	Key	Type	Notes
`_id`	PK	ObjectId, auto-generated	Primary key
`name`		String, required	FR1.1.1
`email`	UK	String, required, unique, lowercase	FR1.1.1
`password`		String, required, select: false	Store the **bcrypt hash** only
`role`		String enum ['admin', 'customer'], default 'customer'	FR1.1.7, FR1.8.1
`contactNumber`		String	FR1.1.3
`profilePicture`		{ url, publicId }	FR1.1.3
`validId`		{ url, publicId }	FR1.1.3. Used for installment applications
`proofOfIncome`		{ url, publicId }	FR1.1.3. Used for installment applications
`isActive`		Boolean, default true	FR1.1.5. Account deactivation
`resetPasswordToken`		String (hashed), select: false	FR1.1.4. Link sent by email
`resetPasswordExpire`		Date, select: false	FR1.1.4

Keys and connections

Primary key	`_id` (ObjectId, auto-generated)
Foreign keys	None
Referenced by	`addresses.user`, `carts.user`, `orders.user`, `orders.installment.application.reviewedBy`, `orders.installment.defaultedBy`, `orders.statusHistory.changedBy`, `payment_schedules.user`, `payments.user`, `payments.recordedBy`, `reviews.user`, `restocks.recordedBy`, `inventory_transactions.performedBy`, `notifications.recipient`
Unique	`email`

Indexes: `{ email: 1 }` unique

> **Note** Customers can have **multiple addresses**, so addresses are stored in the `addresses` collection (4.2) instead of being embedded here. The address given at registration (FR1.1.1) becomes the first saved address.

4.2 addresses

Covers: FR1.1.1, FR1.1.3, FR1.5.2. Saved shipping addresses. A user can have many, and one can be marked as the default.

Field	Key	Type	Notes
`_id`	PK	ObjectId, auto-generated	Primary key
`user`	FK → users	ObjectId (users), required	Owner of the address
`label`		String	e.g. Home, Work
`recipientName`		String, required	Person receiving the delivery
`phoneNumber`		String, required	Delivery contact number (FR1.5.2)
`street`		String, required	House number, street, barangay or subdivision
`city`		String, required	
`postalCode`		String, required	
`country`		String, required	
`isDefault`		Boolean, default false	At most one default per user

Keys and connections

Primary key	`_id` (ObjectId, auto-generated)
Foreign keys	`user → users`
Referenced by	`orders.shippingAddress`
Unique	`{ user }` where `isDefault` is true (one default address per user)

Indexes: `{ user: 1 }`; partial unique `{ user: 1 }` where `isDefault` is true, so a user can have only one default address

> **Note** Orders do not depend on this collection for history. At checkout the chosen address is **copied** into `orders.shippingInfo` and its id is kept in `orders.shippingAddress`, so editing or deleting a saved address never changes past orders.

4.3 categories

Covers: FR1.3.1, FR1.3.2, FR1.3.5. Examples: Motorcycles, Engine Parts, Brakes, Tires and Wheels, Helmets, Riding Gear, Accessories, Lubricants.

Field	Key	Type	Notes
`_id`	PK	ObjectId, auto-generated	Primary key
`name`	UK	String, required, unique	
`description`		String	

Keys and connections

Primary key	`_id` (ObjectId, auto-generated)
Foreign keys	None
Referenced by	`products.category`
Unique	`name`

4.4 brands

Covers: FR1.3.3, FR1.3.4. Examples: Honda, Yamaha, Suzuki, Kawasaki, CFMOTO.

Field	Key	Type	Notes
`_id`	PK	ObjectId, auto-generated	Primary key
`name`	UK	String, required, unique	
`logo`		{ url, publicId }	Optional

Keys and connections

Primary key	`_id` (ObjectId, auto-generated)
Foreign keys	None
Referenced by	`products.brand`
Unique	`name`

4.5 products

Covers: FR1.2.1 to FR1.2.10, FR1.6.6. Catalog items. Average rating and review count are not stored; they come from `reviews`.

Field	Key	Type	Notes
`_id`	PK	ObjectId, auto-generated	Primary key
`name`		String, required	
`description`		String	
`category`	FK → categories	ObjectId (categories), required	FR1.3.2
`brand`	FK → brands	ObjectId (brands), required	
`compatibleModels`		[String]	e.g. "Honda Click 125". Used by FR1.2.5 search
`price`		Number, required, min 0	
`stock`		Number, required, min 0, default 0	Maintained state, updated with every inventory transaction (FR1.6.4)
`lowStockThreshold`		Number	Falls back to the default in `settings` when absent
`images`		[{ url, publicId }]	FR1.2.6. Multiple images
`installmentEligible`		Boolean, default false	FR1.2.3
`minInstallmentPrice`		Number	Minimum price to qualify for installment (FR1.2.3)
`eligiblePlans`	FK → installment_plans	[ObjectId (installment_plans)]	Optional (FR1.2.1)
`isArchived`		Boolean, default false	FR1.2.4

Keys and connections

Primary key	`_id` (ObjectId, auto-generated)
Foreign keys	`category` → `categories`, `brand` → `brands`, `eligiblePlans[]` → `installment_plans`
Referenced by	`carts.items.product`, `orders.items.product`, `reviews.product`, `restocks.items.product`, `inventory_transactions.product`, `notifications.product`

Indexes: text index on `name` and `description`; compound `{ category, brand, price }`; `{ compatibleModels }`

4.6 carts

Covers: FR1.4.1 to FR1.4.5. One cart per user, persisted across sessions. No prices are stored; the current price is read with `$lookup`.

Field	Key	Type	Notes
`_id`	PK	ObjectId, auto-generated	Primary key
`user`	FK → users, UK	ObjectId (users), required, unique	FR1.4.5
`items`	FK → products	[{ product: ObjectId (products), quantity: Number, min 1 }]	FR1.4.1 to FR1.4.3

Keys and connections

Primary key	`_id` (ObjectId, auto-generated)
Foreign keys	`user` → `users`, `items.product` → `products`
Referenced by	None
Unique	`user` (one cart per user)

4.7 orders

Covers: FR1.5.1 to FR1.5.11, FR1.10.3. Order date comes from `createdAt`. No subtotal or total fields are stored.

Field	Key	Type	Notes
`_id`	PK	ObjectId, auto-generated	Primary key
`user`	FK → users	ObjectId (users), required	
`items`	FK → products	[{ product, name, unitPrice, quantity }]	**Snapshot** at checkout (FR1.5.4)
`shippingInfo`		{ address, city, phoneNumber, postalCode, country }, all required	FR1.5.2. **Snapshot** copied from the chosen address
`shippingAddress`	FK → addresses	ObjectId (addresses)	Which saved address the customer picked. Optional, because the snapshot above is what the order uses
`shippingPrice`		Number, default 0	Input value, not computed
`taxRate`		Number	Snapshot of the rate applied at checkout
`paymentMethod`		enum ['cod', 'online', 'installment']	FR1.5.5

Field	Key	Type	Notes
`status`		enum ['For Approval', 'Awaiting Downpayment', 'Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled']	FR1.5.6. The first two apply to installment orders only
`statusHistory`	FK → users	[{ status, changedAt, changedBy: ObjectId (users) }]	Optional audit trail
`deliveredAt`		Date	Set when the status becomes Delivered
`installment`	FK (see below)	Embedded subdocument (below)	Present only if paymentMethod is installment
`cancellation`		Embedded subdocument (below)	Present only if the order was cancelled

installment subdocument

Field	Key	Type	Notes
`plan`	FK → installment_plans	ObjectId (installment_plans)	FR1.5.11
`planSnapshot`		{ name, termMonths, interestRate, minDownpaymentPercent, processingFee }	Frozen copy so later plan edits do not change this contract
`downpayment`		Number	Customer-chosen amount (FR1.5.11)
`downpaymentConfirmedAt`		Date	Set by Admin. Required before Processing (FR1.5.6)
`application`	FK → users	{ status: enum ['For Approval', 'Approved', 'Rejected'], reviewedBy: ObjectId (users), reviewedAt: Date, rejectionReason: String }	FR1.10.3
`validId`		{ url, publicId }	Documents submitted with this application
`proofOfIncome`		{ url, publicId }	
`defaultedAt`		Date	FR1.10.9. Absent means not defaulted
`defaultedBy`	FK → users	ObjectId (users)	Admin who marked the default

cancellation subdocument

Field	Key	Type	Notes
`cancelledAt`		Date	
`reason`		String	
`downpaymentOutcome`		enum ['none', 'refunded', 'forfeited']	FR1.5.9
`refundedAmount`		Number	Recorded according to store policy

Keys and connections

Primary key	`_id` (ObjectId, auto-generated)
Foreign keys	`user` → `users`, `shippingAddress` → `addresses`, `items.product` → `products`, `installment.plan` → `installment_plans`, `installment.application.reviewedBy` → `users`, `installment.defaultedBy` → `users`, `statusHistory.changedBy` → `users`
Referenced by	`payment_schedules.order`, `payments.order`, `inventory_transactions.order`, `notifications.order`

Indexes: `{ user, createdAt: -1 }`, `{ status }`, `{ items.product }`, `{ installment.application.status }`

4.8 installment_plans

Covers: FR1.10.1. Archive plans instead of deleting them so old orders keep a valid reference.

Field	Key	Type	Notes
`_id`	PK	ObjectId, auto-generated	Primary key
`name`		String, required	e.g. "12 Months Standard"
`termMonths`		Number, required	3, 6, 12, 24 or 36
`interestRate`		Number, required	Decide in code whether this is flat or annual and apply it consistently
`minDownpaymentPercent`		Number, required	
`processingFee`		Number, default 0	Optional
`isArchived`		Boolean, default false	

Keys and connections

Primary key	`_id` (ObjectId, auto-generated)
Foreign keys	None
Referenced by	`products.eligiblePlans[]`, `orders.installment.plan`

4.9 payment_schedules

Covers: FR1.10.4, FR1.10.6, FR1.10.7. One document per due date, created when an installment order is approved.

Field	Key	Type	Notes
`_id`	PK	ObjectId, auto-generated	Primary key
`order`	FK → orders	ObjectId (orders), required	
`user`	FK → users	ObjectId (users), required	Convenience for customer queries and reminder emails
`installmentNo`		Number, required	1, 2, 3, and so on
`dueDate`		Date, required	
`amountDue`		Number, required	Fixed contractual amount at approval time, so the agreed schedule never shifts if plans or settings change
`penalty`		{ amount: Number, appliedAt: Date }	Added only when late fees are enabled and the grace period has passed
`reminderSentAt`		Date	For upcoming and missed due-date emails

Keys and connections

Primary key	`_id` (ObjectId, auto-generated)
Foreign keys	`order` → `orders`, `user` → `users`
Referenced by	`payments.allocations.schedule`
Unique	`{ order, installmentNo }`

Indexes: `{ order, installmentNo }` unique; `{ dueDate }`

> **Note Paid, Unpaid and Overdue are not stored.** A row is Paid when the payments allocated to it cover `amountDue`, and Overdue when `dueDate` has passed and it is not Paid. The same applies to the order-level Unpaid, Partially Paid and Paid. Only **Defaulted** is stored (`orders.installment.defaultedAt`) because it is an admin decision.

4.10 payments

Covers: FR1.5.5, FR1.10.5, FR1.10.8, FR1.8.5. All payments: cash on delivery, online, downpayments, scheduled installments, early payoffs and manual admin entries.

Field	Key	Type	Notes
`_id`	PK	ObjectId, auto-generated	Primary key
`order`	FK → orders	ObjectId (orders), required	
`user`	FK → users	ObjectId (users), required	The paying customer
`type`		enum ['full', 'downpayment', 'installment', 'payoff']	
`amount`		Number, required	Amount actually paid
`method`		enum ['cod', 'online', 'cash', 'bank_deposit']	
`referenceNo`		String	Gateway or bank reference. **Never store card details**
`paidAt`		Date, required	
`allocations`	FK → payment_schedules	[{ schedule: ObjectId (payment_schedules), amount: Number }]	Which schedule rows this payment settled. Several entries cover paying ahead or early payoff
`recordedBy`	FK → users	ObjectId (users)	Set only for manual entries recorded by Admin
`receiptNo`	UK	String, unique	Generated identifier for the receipt (viewable and emailable)

Keys and connections

Primary key	`_id` (ObjectId, auto-generated)
Foreign keys	`order` → `orders`, `user` → `users`, `allocations.schedule` → `payment_schedules`, `recordedBy` → `users`
Referenced by	None
Unique	`receiptNo`

Indexes: `{ order, paidAt }`; `{ receiptNo }` unique; `{ allocations.schedule }`

> **Note** Remaining balance is derived: total cost (from the order and plan snapshot) minus the `$sum` of payments.

4.11 reviews

Covers: FR1.7.1 to FR1.7.6. One review per customer per product; reviewing again edits the existing document.

Field	Key	Type	Notes
`_id`	PK	ObjectId, auto-generated	Primary key
`product`	FK → products	ObjectId (products), required	
`user`	FK → users	ObjectId (users), required	
`rating`		Number, required, min 1, max 5	FR1.7.1
`title`		String, required	FR1.7.2
`body`		String, required	FR1.7.2
`isHidden`		Boolean, default false	Admin moderation (FR1.7.5)

Keys and connections

Primary key	`_id` (ObjectId, auto-generated)
Foreign keys	`product` → `products`, `user` → `users`
Referenced by	None
Unique	`{ product, user }` (one review per customer per product)

Indexes: `{ product, user }` **unique**, which enforces FR1.7.6

> **Note The Verified Purchase badge is not stored.** At read time, `$lookup` into `orders` and check for a Delivered order from this user containing the product (FR1.7.4).

4.12 suppliers

Covers: FR1.6.2. Suppliers the admin buys stock from.

Field	Key	Type	Notes
`_id`	PK	ObjectId, auto-generated	Primary key
`name`		String, required	
`contact`		{ person, email, phone, address }	
`productLines`		[String]	Supplied product lines
`isActive`		Boolean, default true	

Keys and connections

Primary key	`_id` (ObjectId, auto-generated)
Foreign keys	None
Referenced by	`restocks.supplier`

4.13 restocks

Covers: FR1.6.1, FR1.6.3. No line cost or total cost is stored; total expense is aggregated.

Field	Key	Type	Notes
`_id`	PK	ObjectId, auto-generated	Primary key
`supplier`	FK → suppliers	ObjectId (suppliers), required	
`items`	FK → products	[{ product: ObjectId (products), quantity: Number, unitCost: Number }]	
`restockedAt`		Date, required	Used for expense report date ranges
`referenceNo`		String	Supplier invoice or purchase order number
`recordedBy`	FK → users	ObjectId (users), required	Admin

Keys and connections

Primary key	`_id` (ObjectId, auto-generated)
Foreign keys	`supplier` → `suppliers`, `items.product` → `products`, `recordedBy` → `users`
Referenced by	`inventory_transactions.restock`

Indexes: `{ restockedAt }`; `{ supplier }`

4.14 inventory_transactions

Covers: FR1.6.4, FR1.6.5. Append-only ledger. Documents are never edited.

Field	Key	Type	Notes
`_id`	PK	ObjectId, auto-generated	Primary key
`product`	FK → products	ObjectId (products), required	
`type`		enum ['restock', 'sale', 'reserve', 'release', 'cancellation', 'adjustment']	`reserve` is used when an installment is approved (FR1.6.4)
`quantityChange`		Number, required	Signed: positive adds stock, negative removes it
`order`	FK → orders	ObjectId (orders)	When caused by an order
`restock`	FK → restocks	ObjectId (restocks)	When caused by a restock
`performedBy`	FK → users	ObjectId (users)	
`note`		String	

Keys and connections

Primary key	`_id` (ObjectId, auto-generated)
Foreign keys	`product` → `products`, `order` → `orders`, `restock` → `restocks`, `performedBy` → `users`
Referenced by	None

Indexes: `{ product, createdAt: -1 }`

> **Note** There are no stock-before or stock-after columns because those are derived. `products.stock` is updated in the same operation, using a transaction or an `$inc` guarded by `stock >= quantity` to prevent overselling.

4.15 notifications

Covers: FR1.5.10, FR1.6.6. In-app admin alerts (such as low stock) and a log of emails sent through Mailtrap.

Field	Key	Type	Notes
`_id`	PK	ObjectId, auto-generated	Primary key
`recipient`	FK → users	ObjectId (users), required	
`channel`		enum ['in_app', 'email']	
`type`		enum ['order_summary', 'order_status', 'installment_approved', 'installment_rejected', 'downpayment_confirmed', 'due_reminder', 'missed_payment', 'receipt', 'low_stock']	
`subject`		String	
`message`		String	
`order`	FK → orders	ObjectId (orders)	Optional
`product`	FK → products	ObjectId (products)	Used for low-stock alerts
`isRead`		Boolean, default false	

Keys and connections

Primary key	`_id` (ObjectId, auto-generated)
Foreign keys	`recipient` → `users`, `order` → `orders`, `product` → `products`
Referenced by	None

4.16 settings

Covers: FR1.10.7, FR1.10.9, FR1.6.6, FR1.5.9. A single document holding admin-configurable values.

Field	Key	Type	Notes
`_id`	PK	ObjectId, auto-generated	Primary key
`taxRate`		Number	Default rate copied into `orders.taxRate` at checkout
`lowStockDefault`		Number	FR1.6.6. Used when a product has no `lowStockThreshold` of its own
`latePenalty`		{ enabled: Boolean, type: enum ['fixed', 'percentage'], value: Number, gracePeriodDays: Number }	FR1.10.7
`missedPaymentsLimit`		Number	FR1.10.9. Missed payments before the admin can mark an account Defaulted

Field	Key	Type	Notes
`cancellationPolicy`		{ downpaymentRefundable: Boolean }	Optional (FR1.5.9)

Keys and connections

Primary key	`_id` (ObjectId, auto-generated)
Foreign keys	None
Referenced by	None

5. Aggregation Map

Every value below is calculated at read time. None of them is a stored field.

Derived value	How it is derived
Cart line total and cart total	`$lookup` product, `$multiply` price by quantity, `$sum`; add tax from `settings`.
Order items price	`$sum` of `$multiply: ["$items.unitPrice", "$items.quantity"]` using `$unwind` or `$reduce`.
Order total	Items price + `shippingPrice` + (items price x `taxRate`).
Financed amount	Order total minus `installment.downpayment`.
Interest, fees, monthly amount, total installment cost	Computed from the financed amount and `planSnapshot` in code or `$project` (FR1.10.2). The stored `amountDue` on each schedule row is the frozen result.
Remaining balance	Total cost minus the `$sum` of `payments.amount` for the order.
Schedule status (Paid, Unpaid, Overdue)	`$lookup` payments by `allocations.schedule`, compare the sum to `amountDue`, and compare `dueDate` with `$$NOW`.
Order installment status	Sum of payments against total cost gives Unpaid, Partially Paid or Paid. Defaulted if `installment.defaultedAt` exists.
Average rating and review count	`$group` on `reviews` by product with `$avg` and `$sum`, excluding `isHidden: true`.
Verified purchase	`$lookup` orders where `status` is Delivered and `items.product` matches.
Total restock expense	`$sum` of `$multiply: ["$items.quantity", "$items.unitCost"]` from `restocks`, grouped by product or date range.
Low stock list	`$match` with `$expr` comparing `stock` to `$ifNull(lowStockThreshold, defaultThreshold)`.
Sales report	`$match` on a `createdAt` range and valid statuses, `$group` by day or month.
Top products by category	`$unwind` items, `$group` by product, `$lookup` category, `$sort` by quantity.
Installment collections, receivables, overdue	Group `payments` by date; `$lookup` schedules to payments for balances; overdue where `dueDate` is before `$$NOW` and unpaid.
Admin dashboard charts	A combination of the aggregations above.

6. Normalization Notes

MongoDB designs trade some normalization for read performance. Where the design departs from third normal form, it is on purpose.

Level	Status	Comment
First normal form	Strictly no	Embedded arrays and subdocuments (`items`, `images`, `allocations`, `address`) are repeating groups. Treated as child tables, they are in 1NF.
Second normal form	Yes	Single `_id` keys avoid partial dependencies. Composite keys (`order + installmentNo`, `product + user`) have attributes that depend on the whole key. The order item `name` depends only on `product`, kept as a snapshot.
Third normal form	Mostly	Intentional redundancy: `user` on schedules and payments (query convenience), `planSnapshot`, `shippingInfo` and ID documents on the order (historical record), `products.stock` (fast reads and overselling guard).

Strict 3NF For strict 3NF, remove `user` from `payment_schedules` and `payments`, drop the item `name`, and drop `products.stock` in favor of summing `inventory_transactions`. The snapshots are the only departures that should be kept, since they record what was true at the time of sale.

7. Implementation Notes

- Use Mongoose `ref` with `populate()` for simple reads and `$lookup` for reports.
- Wrap order placement, installment approval, payment recording and cancellation in a **MongoDB session transaction**. Each one touches several collections. Transactions need a replica set, which MongoDB Atlas provides by default.
- Hash passwords with bcrypt and use JWT for token-based authentication (FR1.8.3). Do not persist any card data (FR1.8.5).
- Restrict installment approval and manual payment recording to the Admin role with middleware (FR1.8.2).
- Prefer archive flags (`isArchived`, `isActive`) over hard deletes for records already referenced elsewhere.
- MongoDB will not reject a foreign key that points to a missing document. Check that the referenced document exists before saving, and block deleting a document that other collections still point to.
- Setting a default address should unset the previous default in the same operation, so each user keeps only one.